

Pytest Quickstart — Markdown-Friendly Guide

Description

- pytest is a fast, simple testing framework for Python. It:
- Auto-discovers tests using file/function naming conventions.
- Uses plain assert statements with rich failure introspection.
- Provides fixtures for reusable setup/teardown.
- Supports parametrization to run one test with many inputs.

Installation and Setup

- Install pytest (and optional coverage plugin):

```
pip install pytest
# optional:
pip install pytest-cov
```

- Recommended structure:

```
my_project/
├─ my_module/
│   └─ math_utils.py
└─ tests/
    └─ test_math_utils.py
```

- Run tests from the project root:

```
pytest          # run all tests
pytest -v       # verbose output
pytest tests/   # run a folder
pytest tests/test_math_utils.py::test_addition # a single test
```

Concepts

1. Test Discovery & Naming

- Files: start with test_ or end with _test.py (e.g., test_math.py).
- Functions: start with test_ (e.g., test_addition).
- Pytest finds and runs them automatically.

2. Assertions

- Use Python's built-in assert.
- Pytest prints clear diffs and values on failure (no need for unittest methods).

3. Fixtures (Setup/Teardown)

- Reusable functions annotated with @pytest.fixture.
- Inject by naming the fixture as a test function parameter.
- Great for creating test data, temporary resources, or configuration.

4. Parametrization

- Run the same test logic with multiple input sets.
- Use @pytest.mark.parametrize("arg1,arg2,...", [(...), (...), ...]).

5. Running Tests (CLI Essentials)

- pytest — run all.
- -v — verbose.
- Target a file or a specific test via ::.

Examples

Example 1 — Minimal Test File (Discovery + Naming)

File: tests/test_math_basics.py

tests/test_math_basics.py

```
def add(a, b):  
    return a + b  
  
def test_addition():  
    # Pytest discovers this because it starts with "test_"  
    assert add(2, 3) == 5  
    assert add(-1, 1) == 0
```

Explanation:

- The file name starts with test_, and the function starts with test_, so pytest will discover and run it automatically.

Example 2 — Running Specific Tests (CLI)

Run all tests:

```
pytest
```

Run a single file:

```
pytest tests/test_math_basics.py
```

Run one test function:

```
pytest tests/test_math_basics.py::test_addition
```

Explanation:

- The :: syntax narrows execution to a specific test within a file—handy for tight feedback loops.

Example 3 — Assertions with Helpful Output

File: tests/test_strings.py

tests/test_strings.py

```
def test_substring_presence():
    message = "hello pytest"
    assert "pytest" in message # passes

def test_equality_failure_example():
    expected = "HELLO"
    actual = "hello"
    assert actual == expected # will fail with a clear diff
```

Explanation:

- Plain assert is enough. On failure, pytest shows actual vs expected values and diffs for strings/collections.

Example 4 — Using Fixtures (Reusable Setup)

File: tests/test_with_fixture.py

tests/test_with_fixture.py

```
import pytest

@pytest.fixture
def sample_user():
    # Setup phase: return reusable test data
    return {"name": "Alice", "age": 30}

def test_user_name(sample_user):
    # The fixture name matches the parameter; pytest injects the returned dict
    assert sample_user["name"] == "Alice"

def test_user_age(sample_user):
    assert sample_user["age"] == 30
```

Explanation:

- @pytest.fixture marks a function as a fixture.
 - Any test that declares sample_user as a parameter will receive the fixture's return value.
 - Use fixtures for setup/teardown without repeating code.
-

Example 5 — Parametrized Tests (Same Logic, Many Inputs)

File: tests/test_parametrized_add.py

tests/test_parametrized_add.py

```
import pytest

def add(a, b):
    return a + b

@pytest.mark.parametrize("a,b,result", [
    (2, 3, 5),
    (-1, 1, 0),
    (10, 5, 15),
    (0, 0, 0),
])
def test_add(a, b, result):
    assert add(a, b) == result
```

Explanation:

- The test test_add runs four times—once per row in the parameter table.
 - Parametrization keeps tests DRY and scales cleanly as you add cases.
-

Example 6 — Combining Fixtures with Parametrization

File: tests/test_combo.py

tests/test_combo.py

```
import pytest

@pytest.fixture
def base():
    # could be a database connection, config object, or complex data
    return 10

@pytest.mark.parametrize("delta,expected", [
    (5, 15),
    (-3, 7),
    (0, 10),
])
def test_base_plus_delta(base, delta, expected):
    assert base + delta == expected
```

Explanation:

- Fixtures **and** parametrization compose nicely: base provides shared setup; para

Example 7 — A Slightly Richer Fixture (Setup/Teardown Pattern)

File: tests/test_setup_teardown.py

```

# tests/test_setup_teardown.py
import pytest

@pytest.fixture
def resource():
    # --- setup ---
    items = []
    yield items # provide to test
    # --- teardown ---
    items.clear() # cleanup logic runs after each test that used this fixture

def test_append(resource):
    resource.append("x")
    assert resource == ["x"]

def test_isolation(resource):
    # Proves each test gets a fresh fixture instance by default (function scope)
    assert resource == []

```

Explanation:

- Using yield in a fixture lets you define setup before yield and teardown after.
- Each test gets an isolated instance (default function scope) unless you change the scope.